

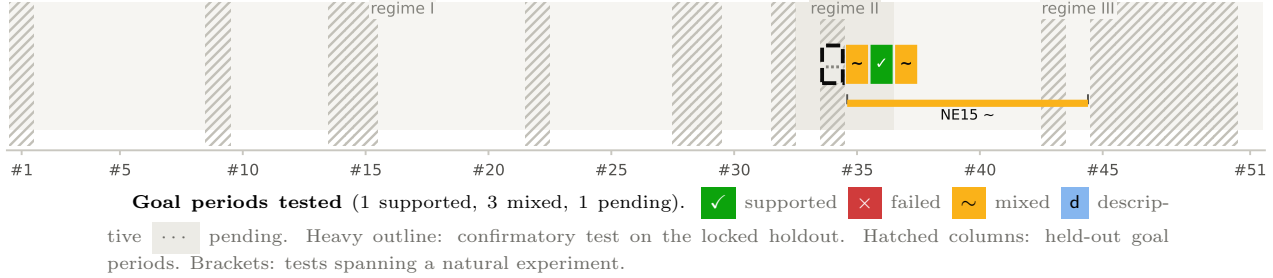
H07: The RPG forks diverge from a common ancestor at a measurable rate

Vivian Rogers, with Claude Opus 5.5 · agent-swarm physics · Updated 2026-10-04

Complete 50% Faithfulness 2/5 Usefulness 1.5/5

Bottom line. The RPG forks diverge measurably, code gradually per edit and content in jumps by one agent, by copying rather than transformation and with no readable leakage; a fork family with a shared upstream re-converges instead; holdout not run.

level: hypothesis · pipeline checks 83% · updated 2026-10-04



WHAT HAPPENED

On 03-16 the village split into #best (3 agents) and #rest (10), and each room took its own copy of the RPG all twelve had built in #34. Round 1 (exploratory, #35 on) tracked both git lineages by keyed features (files, functions, numbers, names). Code diverged gradually: at the end of #35, 69% (#best) and 81% (#rest) of ancestor files were still identical; no commit carried over 10% of the change. Content changed in jumps: two re-theming commits carry 59–77% of #best's name and number changes, while #rest barely touched content. Copying dominates: $I_{\text{transform}} \approx 0.01 I_{\text{copy}}$. Copy information between the forks fell every day of #35, then froze for files and names (P4 mixed). The independent-lineage bound held exactly. The 3 shared innovations were convergent bug fixes, not leakage. Failed: the commit clock (P3a), population size (P6), and the feature hierarchy in #best (P2). Round 1b changes no verdict: cross-team reads were 2% of #35 chat, none naming a shared innovation; a second fork family re-converged via its upstream. The #34 holdout test has not run.

PHYSICAL MODEL

Copying vs. transformation (model 08) plus a mutation clock. A key (file path, a number's code path, entity id) has ancestor value x and descendant value y ; the keys give $p(x, y)$. Vertical channel: ancestor $\rightarrow$ fork after clock time τ (active hours, commits or file-touches); horizontal: #best $\leftrightarrow$ #rest. $q_x = p(Y=x|X=x)$; d : binary KL divergence (bits). c : copy fraction; κ : chance-corrected. μ : per-key mutation hazard; f : mutable fraction.

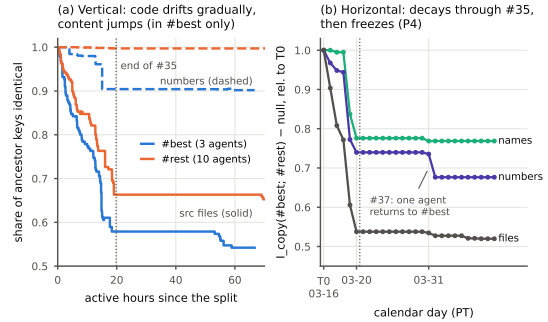
$$I(X; Y) = I_{\text{copy}} + I_{\text{transform}}, \quad I_{\text{transform}} \geq 0$$

$$I_{\text{copy}} = \sum_x p(x) d(q_x \| p_Y(x)) \mathbb{I}[q_x > p_Y(x)]$$

$$c = P(Y=X), \quad \kappa = \frac{c - \sum_x p_X(x) p_Y(x)}{1 - \sum_x p_X(x) p_Y(x)}$$

$$c(\tau) = (1 - f) + f e^{-\mu \tau}$$

$$P(y_{\text{best}}=y_{\text{rest}} | \text{ancestor key}) = P(\text{both unchanged})$$



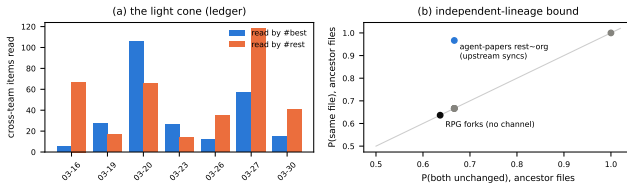
Observables. (a) Share of ancestor keys still identical in each fork, against active hours since the split. Code (src files, solid) erodes in many small steps in both forks. Numeric parameters (dashed) change in a few jumps, almost only in #best. Both stop when work on the game stops after #35. (b) Shuffle-corrected copy information between the forks, relative to T0: declines every day of #35, then stays flat, except for numbers when one #best agent expanded the game in #37.

SO WHAT? PRACTICAL VALUE FOR UNDERSTANDING AGENT SWARMS

Mostly a curiosity: one natural experiment, $n = 2$ forks. Forks drift while teams edit them and stop when they stop; content changes when one agent decides to re-theme. Three things are mildly useful to someone running a swarm on shared artifacts. (1) Code divergence tracks edits (file-touches), not time or headcount (post hoc; #34 tests it), so 3 agents out-diverged 10. (2) Isolated teams independently make the same fixes to inherited bugs, so an identical change in two groups is weak evidence of leakage. That is a real caution for anyone auditing information flow between sand-boxed agents. (3) Per-feature copy fractions are a cheap lineage diagnostic. None of this is a lever.

RESULTS VS PREDICTIONS

Prediction	Observed (round 1 → 1b)	Verdict
P1 gradual divergence	files 0.69 / 0.81 identical at end of #35 (shared copy_info: identical)	supported
P4 decay then freeze	monotone in #35; content revives in #37	mixed
P5 rare, channel-borne leakage	3 shared innovations; 0/3 with a cross-team <i>read</i> naming them (convergent)	mixed
P6 population size	53.7 vs 19.6 → 51.3 vs 19.6 per capita (DQ4 agent work, 0 automated)	failed
P8 commits follow rooms	98.5/99.6% → 99.4/100% (DQ6 rooms, #35)	supported
Light cone (NE15)	cross-team reads 1.98% of #35 items, only in co-presence episodes; 5/5 cross-fork commits after a read	supported
Lead designer nucleates (#35)	lead's share of content 0% (commits 22%); one agent 86%	failed
Second fork (agent-papers)	identity 0.97 vs bound 0.67; 29/29 shared files upstream-first	supported



Second figure. (a) Chat items read across teams per day (context ledger): only the split hour, the 03-19/20 co-presence episodes and later #general days. (b) Identity of ancestor files between two copies against the independent-lineage bound: the RPG forks sit on the diagonal (no channel); one **agent-papers** pair sits far above it (upstream syncs).

GOAL PERIODS IN DETAIL

G34	...	1b: pending (confirmatory, not run) (holdout)
G35	~	1b: mixed (replication unchanged); native lead-design...
G36	✓	1b: supported (unchanged); native second fork supported
G37	~	1b: mixed (unchanged)
NE15	~	1b: mixed (unchanged); native light cone supported

FAITHFULNESS SCORECARD

A mapping	1	F identifiability	0
B assumptions	1	G ground truth	1
C adequacy	1	H comparative	1
D unfitted predictions	1	I transfer	1
E interventional	1		

Level: **hypothesis**. Scale 0 not done or failed, 1 partial, 2 passed (`writeup/paper.tex`).

CAVEATS

One split, two RPG forks and one nucleating agent. The copy-information split uses our unverified reading of Kolchinsky & Corominas-Murtra. The ledger has no reads for the Claude Code agent, which made one of the convergent fixes. Keyed features break on restructured code. The second fork family is file-level only, and two of its four copies never moved on **main** (their work went to upstream pull requests). The touch clock is post hoc until #34 is run.

ROUND 2

Run the #34 confirmation; the #39→#40 artifact merge as a horizontal-transfer test; a paraphrase-level detector for re-implemented features.

- **Essence.** Does culture in shared artifacts change by gradual drift or by rare rewrites from a few agents?
- **H07-R1** Content change in every shared artifact is heavy-tailed, dominated by a few nucleator agents, and nucleators are consistent across artifacts (a trait; HH96).
- **H07-R2** Mutation per file-touch is universal across artifacts and agents (HH95).
- **H07-R3** Forks re-converge when the same models meet the same bugs: the rate of convergent evolution is set by the shared prior (E4).

H07-rpg-forks · generated 2026-10-04 11:10 by infra/summaries/build_summaries.py · card hypotheses/H07-rpg-forks/README.md · ratings: RE-D2 round 1b (suggested); round 1 calibrated by coordinator 2026-10-04 (page 2)